

Understanding the new structure of the project (terraform)

There are really two main types of files in the project:

```
1 .tf files
2   ↓
3 Terraform infrastructure
4   ↓
5 Tell AWS what resources should exist
6
7
8 .mjs files
9   ↓
10 JavaScript application code
11   ↓
12 Tell the Lambda functions what to do
```

Terraform is responsible for things such as:

```
1 API Gateway
2 Lambda
3 DynamoDB
4 SNS
5 SQS
6 IAM
7 CloudWatch
8 API key
```

JavaScript is responsible for:

```
1 Handling API requests
2 Validating readings
3 Generating UUIDs
4 Calculating status
5 Reading/writing DynamoDB
6 Publishing excursion alerts
7 Processing SQS messages
```

The original project already used this distinction between infrastructure and application code.

Current project structure

The project is approximately:

```
1 terraform-week3/
2 |
3 |— lambda/
4 |   |
5 |   |— link-dynamodb/
6 |   |   |
7 |   |   |— controller/
8 |   |   |   └─ readingsController.mjs
9 |   |   |
10 |   |   |— services/
11 |   |   |   └─ readingsService.mjs
12 |   |   |
13 |   |   |— repositories/
14 |   |   |   └─ readingsRepositories.mjs
```

```

15 |     |
16 |     |   └─ index.mjs
17 |     |
18 |     └─ excursion-handler/
19 |         |
20 |         └─ controller/
21 |             └─ excursionController.mjs
22 |         |
23 |         └─ services/
24 |             └─ excursionServices.mjs
25 |         |
26 |         └─ repositories/
27 |             └─ excursionRepositories.mjs
28 |         |
29 |         └─ index.mjs
30 |
31 | └─ terraform/
32 |     |
33 |     └─ main.tf
34 |     └─ variables.tf
35 |     └─ dynamodb.tf
36 |     └─ lambda.tf
37 |     └─ iam.tf
38 |     └─ api-gateway.tf
39 |     └─ api-key.tf
40 |     └─ sns.tf
41 |     └─ sqs.tf
42 |     └─ output.tf
43 |
44 | └─ .gitignore
45 | └─ README.md

```

There are two Lambda applications.

```

1 | link-dynamodb

```

handles normal REST API requests.

```

1 | excursion-handler

```

handles asynchronous excursion alerts coming from SQS.

The main Lambda

The first Lambda is:

```

1 | umar-coldchain-LinkDynamoDB-terraform

```

Its code lives inside:

```

1 | lambda/
2 |   └─ link-dynamodb/

```

This Lambda handles:

```

1 | POST
2 | GET all
3 | GET one
4 | PATCH
5 | DELETE

```

It also decides when an excursion should be sent to SNS.

The Lambda is still one AWS Lambda.

Controller-Service-Repository does not create three Lambdas.

It simply separates the code inside one Lambda into different responsibilities.

`index.mjs` — Lambda entry point

The first file AWS starts is:

```
1 | lambda/link-dynamodb/index.mjs
```

Terraform configures:

```
1 | handler = "index.handler"
```

and the JavaScript contains something similar to:

```
1 | import { handleRequest } from "../controller/readingsController.mjs";
2 |
3 | export const handler = async (event, context) => {
4 |   return handleRequest(event, context);
5 | };
```

So:

```
1 | API Gateway
2 |   ↓
3 | Lambda
4 |   ↓
5 | index.mjs
6 |   ↓
7 | readingsController.mjs
```

`index.mjs` deliberately does very little.

Its job is essentially:

AWS has given me an event. Pass it to the controller.

The old project used the same basic idea of keeping the Lambda entry point small and passing work into the controller.

Controller — what request did we receive?

The controller is:

```
1 | lambda/link-dynamodb/controller/readingsController.mjs
```

Its main responsibility is routing.

The current API now supports:

```
1 POST
2 /branches/{branchId}/readings
3
4 GET
5 /branches/{branchId}/readings
6
7 GET
8 /branches/{branchId}/readings/{readingId}
9
10 PATCH
11 /branches/{branchId}/readings/{readingId}
12
13 DELETE
14 /branches/{branchId}/readings/{readingId}
```

The controller creates a route value similar to:

```
1 const route =
2   `${event.httpMethod} ${event.resource}`;
```

and then uses:

```
1 switch (route)
```

to decide what operation should run.

Conceptually:

```
1 POST /readings
2   ↓
3   createReading()
4
5
6 GET /readings
7   ↓
8   getBranchReadings()
9
10
11 GET /readings/{readingId}
12   ↓
13   getReadingById()
14
15
16 PATCH /readings/{readingId}
17   ↓
18   patchReading()
19
20
21 DELETE /readings/{readingId}
22   ↓
23   deleteReading()
```

So the controller can still be thought of as the **receptionist**.

It answers:

What is the client asking us to do?

It does not contain DynamoDB commands and should not contain the main temperature business rules.

Service — business logic

The main service is:

```
1 | lambda/link-dynamodb/services/readingsService.mjs
```

This is where the cold-chain rules live.

For example:

```
1 | if (  
2 |   minTempC < 2 ||  
3 |   maxTempC > 8  
4 | ) {  
5 |   return "excursion";  
6 | }  
7 |  
8 | return "ok";
```

The service handles things such as:

```
1 | Are minTempC and maxTempC numbers?  
2 |  
3 | Is maxTempC lower than minTempC?  
4 |  
5 | Is recordedAt valid?  
6 |  
7 | Is recordedAt in the future?  
8 |  
9 | Is recordedBy valid?  
10 |  
11 | Should status be "ok"?  
12 |  
13 | Should status be "excursion"?  
14 |  
15 | Should an SNS alert be published?  
16 |  
17 | Can this PATCH field be modified?  
18 |  
19 | Does the reading belong to this branch?
```

This is also where the new UUID behaviour happens.

UUIDv4 is generated by the server

This is one of the major changes from the earlier project.

The client no longer sends:

```
1 | {  
2 |   "readingId": "some-uuid",  
3 |   "recordedAt": "...",  
4 |   "minTempC": 3,  
5 |   "maxTempC": 7,
```

```
6   "recordedBy": "Umar"
7 }
```

Instead the client sends:

```
1 {
2   "recordedAt": "2026-08-31T14:00:00.000Z",
3   "minTempC": 3,
4   "maxTempC": 7,
5   "recordedBy": "Umar"
6 }
```

Inside the service we use Node's built-in UUIDv4 generator:

```
1 import { randomUUID } from "node:crypto";
```

and when POST creates the reading:

```
1 const readingId =
2   randomUUID();
```

So:

```
1 POST request
2   ↓
3 Service validates body
4   ↓
5 randomUUID()
6   ↓
7 UUIDv4 created
8   ↓
9 readingId assigned
10  ↓
11 saved to DynamoDB
```

The client therefore doesn't control the identifier.

The server does.

readingId and **recordedAt** are completely separate

The old design used:

```
1 readingKey =
2 recordedAt#readingId
```

For example:

```
1 2026-08-27T09:00:00.000Z#6e8a5804-...
```

That no longer exists.

The active table stores:

```
1 readingId
2 branchId
3 recordedAt
4 minTempC
```

```
5 maxTempC
6 recordedBy
7 status
8 alertRaisedAt
```

For example:

```
1 readingId
2 40669387-918d-4f6f-870a-65d6db4dad2e
3
4 branchId
5 BRANCH-001
6
7 recordedAt
8 2026-08-31T14:00:00.000Z
```

This means two completely different readings can happen at exactly the same time:

```
1 UUID-A → 12:00:00
2 UUID-B → 12:00:00
3 UUID-C → 12:00:00
```

That is valid.

But two records cannot have the same:

```
1 readingId
```

because `readingId` is the DynamoDB primary key.

Repository — talks to DynamoDB

The repository is:

```
1 lambda/link-dynamodb/repositories/readingsRepositories.mjs
```

Its responsibility is:

Talk to DynamoDB and other external AWS services.

This is where AWS SDK commands live.

For example:

```
1 new PutCommand(...)
```

```
1 new GetCommand(...)
```

```
1 new QueryCommand(...)
```

```
1 new UpdateCommand(...)
```

```
1 new DeleteCommand(...)
```

The service shouldn't need to understand the exact DynamoDB request.

Instead it can say:

```
1 | await findReadingById(readingId);
```

or:

```
1 | await saveReading(item);
```

or:

```
1 | await deleteReadingFromDatabase(readingId);
```

The repository then performs the actual AWS SDK operation.

So:

```
1 | Service
2 |   ↓
3 | "Find this reading"
4 |
5 | Repository
6 |   ↓
7 | GetCommand
8 |
9 | DynamoDB
```

The Controller-Service-Repository separation was also the central idea of the previous structure.

New DynamoDB structure

The active DynamoDB table is:

```
1 | umar-coldchain-readings-terraform
```

The primary key is:

```
1 | Partition key:
2 | readingId
```

There is no sort key on the base table.

Conceptually:

```
1 | DynamoDB
2 | _____
3 |
4 | readingId      PRIMARY KEY
5 | branchId
6 | recordedAt
7 | minTempC
8 | maxTempC
9 | recordedBy
10 | status
11 | alertRaisedAt
12 |
13 | _____
```

Because `readingId` is the primary key, DynamoDB naturally prevents two items with the same ID when our conditional write is used.

The repository also uses:

```
1 | ConditionExpression:  
2 | "attribute_not_exists(readingId)"
```

when creating a reading.

That is an additional protection against accidentally overwriting an existing reading.

Why do we have a DynamoDB GSI?

There is an important problem with making:

```
1 | readingId
```

the primary key.

It makes finding one reading extremely easy:

```
1 | readingId  
2 |   ↓  
3 | GetItem
```

But what if we want:

```
1 | All readings for BRANCH-001
```

We don't know all their UUIDs.

That is why the table also has a **Global Secondary Index — GSI**.

The GSI uses:

```
1 | Partition key:  
2 | branchId  
3 |  
4 | Sort key:  
5 | recordedAt
```

Conceptually:

```
1 | Main DynamoDB table  
2 |  
3 | readingId  
4 |   ↓  
5 | find ONE reading  
6 |  
7 |  
8 | GSI  
9 |  
10 | branchId + recordedAt  
11 |   ↓  
12 | find readings belonging to a branch  
13 |   ↓
```

```
14 sort/filter them by time
```

This allows:

```
1 GET all branch readings
2
3 GET readings between two dates
4
5 GET newest readings first
```

while still allowing multiple readings with exactly the same `recordedAt` .

POST flow

Suppose Postman sends:

```
1 {
2   "recordedAt": "2026-08-31T14:00:00.000Z",
3   "minTempC": 3,
4   "maxTempC": 7,
5   "recordedBy": "Umar"
6 }
```

The flow is:

```
1 Postman
2   ↓
3 POST /branches/BRANCH-001/readings
4   ↓
5 API Gateway
6   ↓
7 LinkDynamoDB Lambda
8   ↓
9 index.mjs
10  ↓
11 readingsController.mjs
12  ↓
13 createReading()
14  ↓
15 readingsService.mjs
16  |
17  ├── validate input
18  ├── validate timestamp
19  ├── calculate status
20  └── generate UUIDv4
21  ↓
22 readingsRepositories.mjs
23  ↓
24 PutCommand
25  ↓
26 DynamoDB
```

The response could be:

```
1 {
2   "readingId": "40669387-918d-4f6f-870a-65d6db4dad2e",
3   "branchId": "BRANCH-001",
4   "recordedAt": "2026-08-31T14:00:00.000Z",
5   "minTempC": 3,
6   "maxTempC": 7,
7   "recordedBy": "Umar",
```

```
8   "status": "ok"
9 }
```

The important point is:

```
1 Client sends no readingId
2
3 Server generates readingId
```

GET all readings flow

Request:

```
1 GET
2 /branches/BRANCH-001/readings
```

Flow:

```
1 API Gateway
2   ↓
3 Controller
4   ↓
5 getBranchReadings()
6   ↓
7 Service
8   ↓
9 Repository
10  ↓
11 QueryCommand
12  ↓
13 branch-recordedAt-index
14  ↓
15 DynamoDB
16  ↓
17 200 OK
```

Optional query parameters can also provide:

```
1 from
2 to
```

so the GSI can query a time range.

GET one reading flow

Request:

```
1 GET
2 /branches/BRANCH-001/readings/{readingId}
```

Because `readingId` is the DynamoDB primary key:

```
1 readingId
2   ↓
3 GetCommand
4   ↓
5 DynamoDB
```

The service then verifies:

```
1 Does this reading exist?
2
3 Does its branchId equal BRANCH-001?
```

If yes:

```
1 200 OK
```

If not:

```
1 404 Reading not found
```

PATCH flow

Suppose the current item is:

```
1 {
2   "readingId": "uuid",
3   "minTempC": 3,
4   "maxTempC": 7,
5   "status": "ok"
6 }
```

The client sends:

```
1 {
2   "maxTempC": 12
3 }
```

to:

```
1 PATCH
2 /branches/BRANCH-001/readings/{readingId}
```

The flow becomes:

```
1 PATCH
2   ↓
3 Controller
4   ↓
5 patchReading()
6   ↓
7 Service
8   ↓
9 Find existing reading
10  ↓
11 old:
12 min = 3
13 max = 7
14
15 new:
16 min = 3
17 max = 12
18   ↓
19 Recalculate status
20   ↓
21 status = excursion
```

```
22   ↓
23 Repository
24   ↓
25 UpdateCommand
26   ↓
27 DynamoDB
```

PATCH only allows specific fields such as:

```
1 | minTempC
2 | maxTempC
3 | recordedBy
```

Things such as:

```
1 | readingId
2 | branchId
3 | recordedAt
4 | status
5 | alertRaisedAt
```

cannot be directly changed by the client.

If PATCH changes a reading from:

```
1 | ok
```

to:

```
1 | excursion
```

the service publishes an SNS event.

DELETE flow

Request:

```
1 DELETE
2 /branches/BRANCH-001/readings/{readingId}
```

Flow:

```
1 Controller
2   ↓
3 deleteReading()
4   ↓
5 Service
6   ↓
7 Find reading by UUID
8   ↓
9 Does it exist and belong to branch?
10  ↓
11 YES
12  ↓
13 Repository
14  ↓
15 DeleteCommand
16  ↓
17 DynamoDB
```

```
18   ↓
19 204 No Content
```

If the ID doesn't exist:

```
1 404 Reading not found
```

Excursion alert flow

Suppose a POST or PATCH creates:

```
1 status = excursion
```

The main Lambda then publishes an event:

```
1 LinkDynamoDB Lambda
2   ↓
3   SNS
4   ↓
5   SQS
6   ↓
7 Excursion Handler Lambda
8   ↓
9   DynamoDB
10  ↓
11 alertRaisedAt
```

This is asynchronous.

The API does not need to sit waiting for the excursion Lambda to complete its work.

SNS

Terraform file:

```
1 sns.tf
```

creates:

```
1 umar-coldchain-excursions-terraform
```

SNS is used to publish excursion events.

The JavaScript service decides:

```
1 Should an excursion event be published?
```

but AWS SNS itself is infrastructure.

Therefore we don't create:

```
1 snsController.mjs
2 snsService.mjs
```

Terraform creates and configures SNS.

AWS runs SNS.

The previous project notes made the same important distinction between SNS/SQS infrastructure and JavaScript Lambda code.

SQS

Terraform file:

```
1 | sqs.tf
```

creates:

```
1 | umar-coldchain-alert-queue-terraform
```

and:

```
1 | umar-coldchain-alert-dlq-terraform
```

The flow is:

```
1 | SNS
2 | ↓
3 | Main SQS queue
4 | ↓
5 | Excursion Handler Lambda
```

If the excursion Lambda repeatedly fails:

```
1 | SQS
2 | ↓
3 | retry
4 | ↓
5 | retry
6 | ↓
7 | retry
8 | ↓
9 | DLQ
```

The DLQ protects failed messages so they aren't simply lost.

Excursion Handler Lambda

The second Lambda is:

```
1 | umar-coldchain-excursion-handler-terraform
```

Its code lives in:

```
1 | lambda/excursion-handler/
```

Unlike the first Lambda, this one doesn't receive HTTP requests.

It receives:

```
1 | SQS events
```

So:

```
1 | SQS
2 | ↓
3 | excursion-handler/index.mjs
4 | ↓
5 | excursionController.mjs
6 | ↓
7 | excursionServices.mjs
8 | ↓
9 | excursionRepositories.mjs
10 | ↓
11 | DynamoDB
```

The old notes described the same distinction: the main Lambda is API-driven, while the excursion Lambda is SQS-driven.

Excursion Controller

The excursion controller receives:

```
1 | event.Records
```

from SQS.

It:

```
1 | reads SQS records
2 | ↓
3 | parses JSON
4 | ↓
5 | passes message to service
```

It also uses a `switch` on the result returned by the service, such as:

```
1 | raised
2 | duplicate
```

This is a good use of `switch` because there are several known possible values.

Excursion Service

The service controls excursion-specific rules.

For example:

```
1 | forceFailure = true?
2 | ↓
3 | throw deliberate failure
```

This is useful for testing:

```
1 | SQS retries
```

Otherwise the service asks the repository to add:

```
1 alertRaisedAt
```

to the DynamoDB reading.

Excursion Repository

The repository performs the DynamoDB update.

It uses a condition similar to:

```
1 attribute_not_exists(alertRaisedAt)
```

This gives us **idempotency**.

SQS provides at-least-once delivery, meaning the same message can sometimes be delivered more than once.

Without protection:

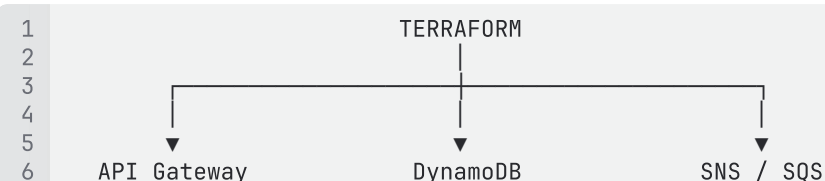
```
1 same message
2   ↓
3 alert raised
4   ↓
5 message delivered again
6   ↓
7 alert raised again ❌
```

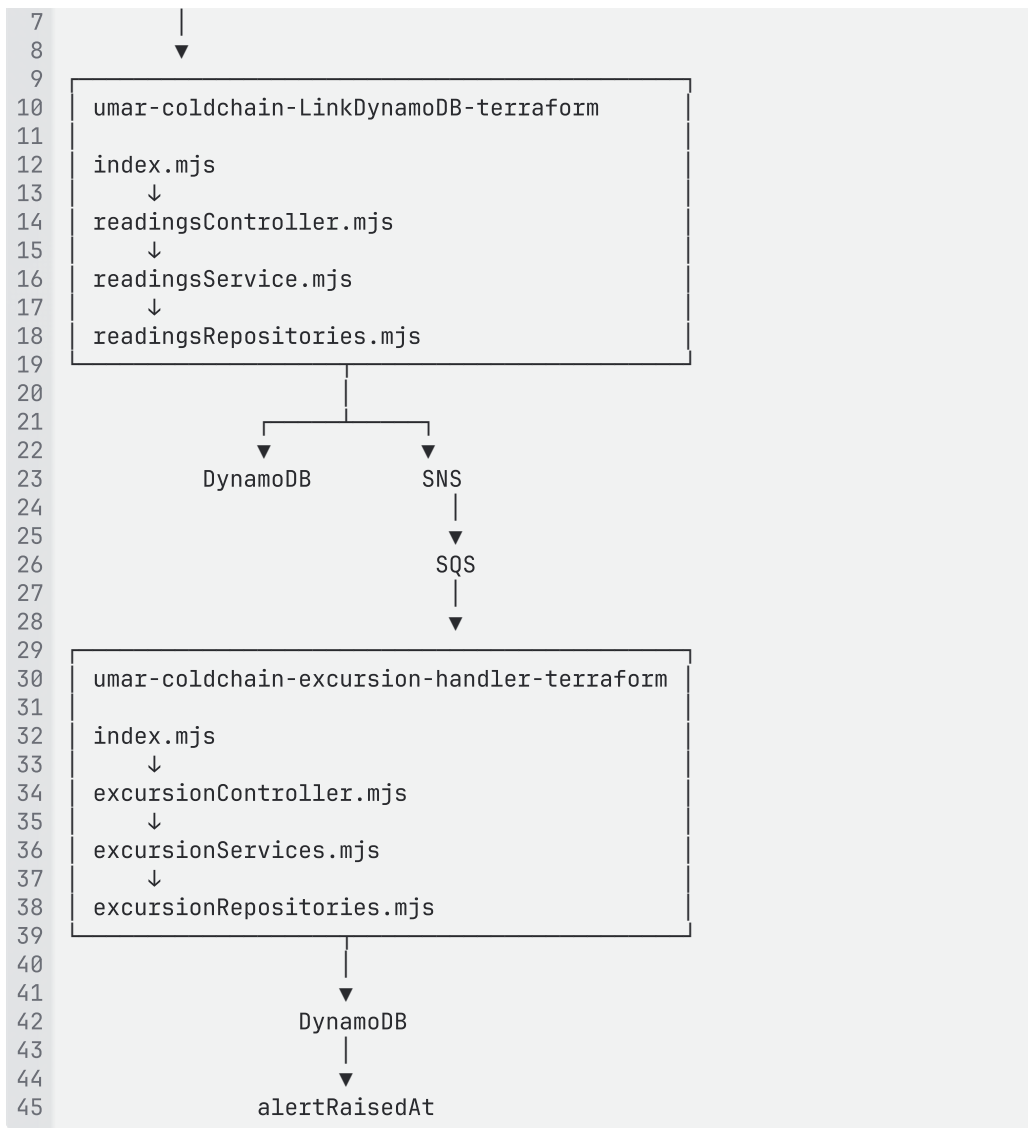
With the condition:

```
1 first message
2   ↓
3 alertRaisedAt does not exist
4   ↓
5 set alertRaisedAt
6
7
8 duplicate message
9   ↓
10 alertRaisedAt already exists
11  ↓
12 do not raise duplicate alert
```

Complete application flow

The full architecture can now be understood like this:





What each Terraform file does

main.tf

This tells Terraform which providers are required.

For example:

```
1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5     }
6   }
7 }
```

and:

```
1 provider "aws" {
2   region = var.aws_region
3 }
```

It effectively says:

We are using AWS, and Terraform should communicate with AWS in the configured region.

variables.tf

Variables are **inputs into Terraform**.

For example:

```
1 variable "api_name" {  
2     type    = string  
3     default = "umar-coldchain-api-terraform"  
4 }
```

Terraform can then use:

```
1 var.api_name
```

instead of repeating:

```
1 umar-coldchain-api-terraform
```

throughout the project.

Variables store things such as:

```
1 AWS region  
2 API name  
3 Lambda names  
4 DynamoDB table name  
5 SNS topic name  
6 SQS names  
7 timeouts  
8 CloudWatch retention
```

dynamodb.tf

This creates the active table:

```
1 umar-coldchain-readings-terraform
```

Its main key is:

```
1 readingId
```

and it creates the GSI:

```
1 branchId + recordedAt
```

So:

```
1 readingId
2   ↓
3 get one reading
4
5
6 branchId + recordedAt
7   ↓
8 query groups of readings
```

There is no longer a:

```
1 readingKey
```

combining timestamp and UUID.

lambda.tf

This packages the JavaScript folders and creates both Lambda functions.

Conceptually:

```
1 lambda/link-dynamodb/
2   ↓
3   ZIP
4   ↓
5 umar-coldchain-LinkDynamoDB-terraform
```

and:

```
1 lambda/excursion-handler/
2   ↓
3   ZIP
4   ↓
5 umar-coldchain-excursion-handler-terraform
```

It also supplies environment variables such as:

```
1 TABLE_NAME
2 EXCURSION_TOPIC_ARN
3 BRANCH_RECORDED_AT_INDEX
```

That means the JavaScript doesn't need hard-coded resource names.

iam.tf

IAM decides what each Lambda is allowed to do.

The main Lambda needs access roughly like:

```
1 LinkDynamoDB Lambda
2   |
3   ├── DynamoDB GetItem
4   ├── DynamoDB PutItem
5   ├── DynamoDB Query
6   ├── DynamoDB UpdateItem
7   └── DynamoDB DeleteItem
```

```
8 | SNS Publish
9 | CloudWatch logging
```

The excursion Lambda needs:

```
1 | Excursion Handler
2 |
3 |   └─ SQS ReceiveMessage
4 |   └─ SQS DeleteMessage
5 |   └─ SQS GetQueueAttributes
6 |   └─ DynamoDB GetItem
7 |   └─ DynamoDB UpdateItem
8 |   └─ CloudWatch logging
```

This follows the principle of giving each Lambda only the permissions it actually needs.

api-gateway.tf

This creates:

```
1 | umar-coldchain-api-terraform
```

and builds the URL tree:

```
1 | /
2 | └─ branches
3 |   └─ {branchId}
4 |     └─ readings
5 |       └─ {readingId}
```

The routes are:

```
1 | POST   /branches/{branchId}/readings
2 |
3 | GET    /branches/{branchId}/readings
4 |
5 | GET    /branches/{branchId}/readings/{readingId}
6 |
7 | PATCH  /branches/{branchId}/readings/{readingId}
8 |
9 | DELETE /branches/{branchId}/readings/{readingId}
```

API Gateway then connects those methods to:

```
1 | umar-coldchain-LinkDynamoDB-terraform
```

api-key.tf

This creates the API key configuration and usage plan.

The client sends:

```
1 | x-api-key
```

with its requests.

Conceptually:

```
1 Postman
2   ↓
3 x-api-key
4   ↓
5 API Gateway
6   ↓
7 Is key accepted?
8   ↓
9 YES
10  ↓
11 Lambda
```

The API key is not stored directly in source code.

sns.tf

Creates:

```
1 umar-coldchain-excursions-terraform
```

and connects:

```
1 SNS
2   ↓
3 SQS
```

through a subscription.

sqs.tf

Creates:

```
1 umar-coldchain-alert-queue-terraform
```

and:

```
1 umar-coldchain-alert-dlq-terraform
```

It configures:

```
1 visibility timeout
2 redrive policy
3 maximum receive count
4 SNS permission
5 Lambda event-source mapping
```

output.tf

Outputs are **information coming out of Terraform**.

For example:

```
1 output "api_base_url" {
2   value = "...
3 }
```

does not create an API.

Instead it says:

Terraform, once you've created everything, show me the API URL.

So:

```
1 variables
2   ↓
3 INPUT into Terraform
4
5
6 resources
7   ↓
8 things Terraform creates
9
10
11 outputs
12   ↓
13 useful information Terraform gives back
```

Examples include:

```
1 API URL
2 DynamoDB table name
3 Lambda names
4 SNS topic ARN
5 SQS queue URL
6 API key ID
```

The easiest way to understand Terraform resource syntax

Whenever you see:

```
1 resource "aws_lambda_function" "link_dynamodb" {
```

read it as:

```
1 resource
2   ↓
3 Terraform should manage/create something
4
5
6 aws_lambda_function
7   ↓
8 WHAT kind of AWS thing?
9 Lambda
10
11
12 link_dynamodb
13   ↓
14 What do WE call this resource inside Terraform?
```

Similarly:

```
1 resource "aws_dynamodb_table" "readings_by_id" {
```

means:

```
1 Terraform resource
2   ↓
3 AWS DynamoDB table
4   ↓
5 internally called readings_by_id
```

And:

```
1 resource "aws_sqs_queue" "alert_queue" {
```

means:

```
1 Terraform resource
2   ↓
3 AWS SQS queue
4   ↓
5 internally called alert_queue
```

This is how Terraform knows what AWS service to create.

Why Controller-Service-Repository is useful

Without this structure, we could put everything into:

```
1 index.mjs
```

but then one file could contain:

```
1 HTTP routing
2 validation
3 UUID creation
4 temperature rules
5 DynamoDB commands
6 SNS
7 PATCH
8 DELETE
9 error handling
10 logging
```

That becomes difficult to maintain.

Instead:

```
1 CONTROLLER
2 "What request came in?"
3
4   ↓
5
6 SERVICE
7 "What should happen?"
8
9   ↓
10
```

```
11 REPOSITORY
12 "How do I communicate with AWS/DynamoDB?"
```

For example, if:

```
1 maxTempC = 12
```

incorrectly gives:

```
1 status = ok
```

look in:

```
1 readingsService.mjs
```

If the status is correct but nothing is written:

```
1 readingsRepositories.mjs
```

is likely the place to investigate.

If POST reaches Lambda but the route isn't recognised:

```
1 readingsController.mjs
```

is where to investigate.

That debugging benefit was one of the main reasons the original project was split into Controller-Service-Repository.

Simplest way to remember the entire project

Think of **Terraform as constructing the building**:

```
1 Terraform
2   ↓
3 build API Gateway
4 build API key
5 build Lambdas
6 build DynamoDB
7 build SNS
8 build SQS
9 build IAM
10 build CloudWatch configuration
```

Then think of the **JavaScript as the people working inside that building**:

```
1 Controller
2   ↓
3 receives the job
4
5 Service
6   ↓
7 understands the rules
8
9 Repository
10  ↓
```

And there are two main JavaScript applications:

```

1 TEAM 1
2
3 link-dynamodb/
4
5 Handles REST API requests:
6 POST
7 GET
8 PATCH
9 DELETE

```

and:

```

1 TEAM 2
2
3 excursion-handler/
4
5 Handles asynchronous excursion alerts.

```

Between them:

```

1 SNS
2 ↓
3 SQS

```

provides the asynchronous messaging system.

The simplest overall flow to remember is:

```

1 CLIENT
2 ↓
3 API Gateway
4 ↓
5 Main Lambda
6 ↓
7 Controller
8 ↓
9 Service
10 ↓
11 Repository
12 ↓
13 DynamoDB
14
15 If excursion:
16 ↓
17 SNS
18 ↓
19 SQS
20 ↓
21 Excursion Lambda
22 ↓
23 Controller
24 ↓
25 Service
26 ↓
27 Repository
28 ↓
29 DynamoDB
30 ↓
31 alertRaisedAt

```

That is the **current structure of your Terraform cold-chain project**, including the UUIDv4 redesign, separate `readingId` and `recordedAt` , the new DynamoDB access pattern, RESTful PATCH/DELETE routes, API key, and the existing SNS/SQS alert flow.